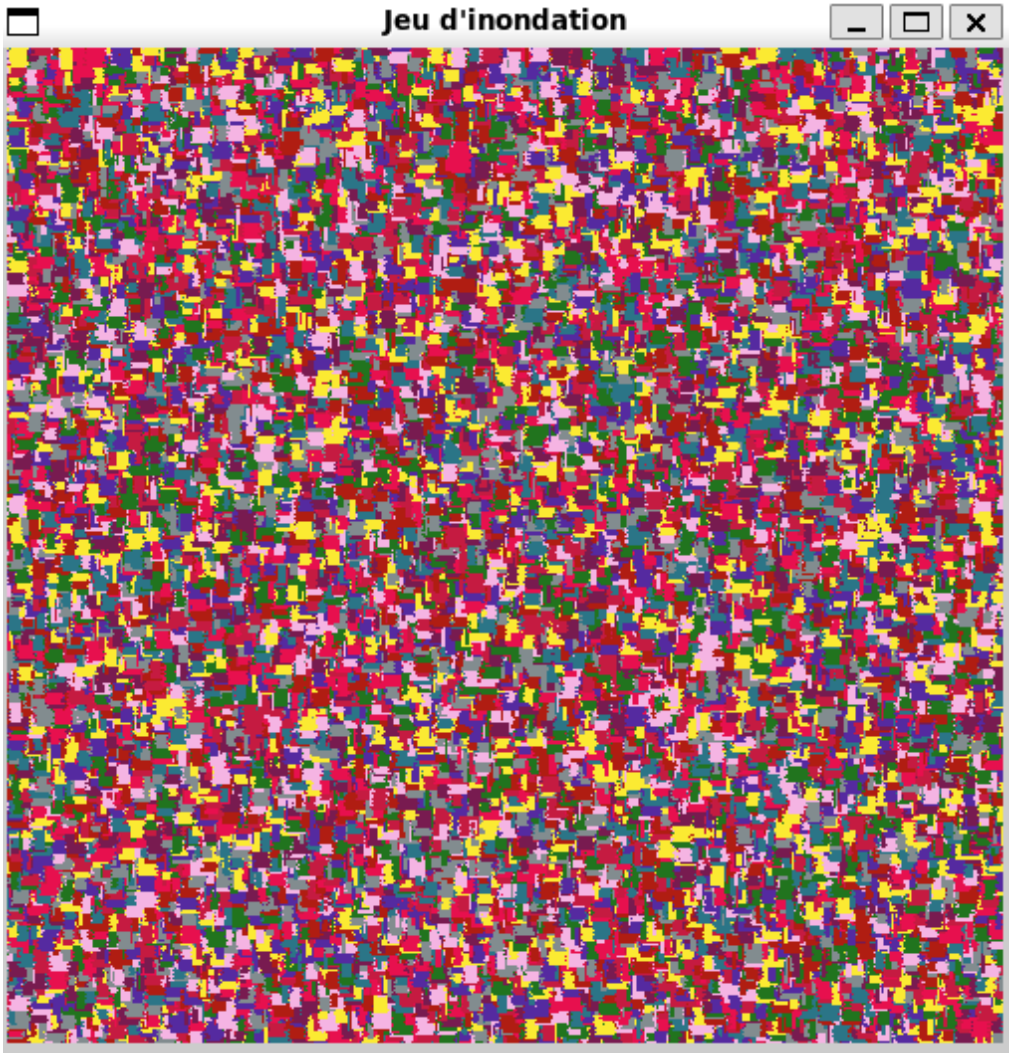
RAPPORT DE PROJET : FLOOD-IT

Auteur : KHIMOUM Aimen Mohamed



1.Introduction

Ce projet a pour objectif d'implémenter et d'analyser différentes stratégies algorithmiques pour résoudre le jeu Flood-It. Le but du jeu est d'inonder une grille de dimensions NxN avec une seule couleur en un nombre minimal de coups, en partant de la zone supérieure gauche.

J'ai implémenté plusieurs versions, allant d'une approche naïve (aléatoire) à des approches déterministes basées sur le gain immédiat (Max-Bordure) et optimisées (Max-Bordure-Zone), pour finir par une stratégie d'anticipation. Ce rapport présente l'analyse de mes résultats, les difficultés rencontrées et une comparaison détaillée des performances.

2.Analyse des Algorithmes et Résultats

2.1. Exercice 1 vs Exercice 2 : Récursivité vs Itératif

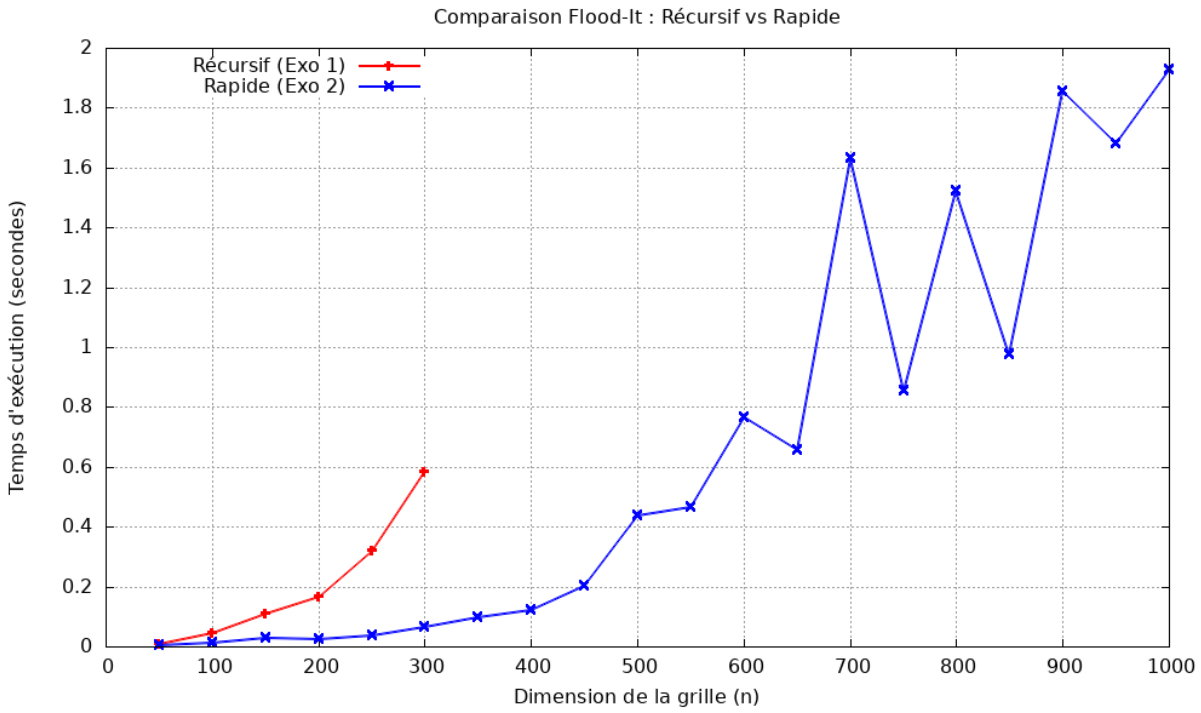
L'exercice 1 utilisait une approche récursive pour identifier la zone Zsg (`trouve_zone_rec`). L'exercice 2 a introduit une structure de données itérative (`S_Zsg`) maintenant une liste de bordure à jour.

Difficulté Majeure : Le **Segmentation Fault (core dumped)**

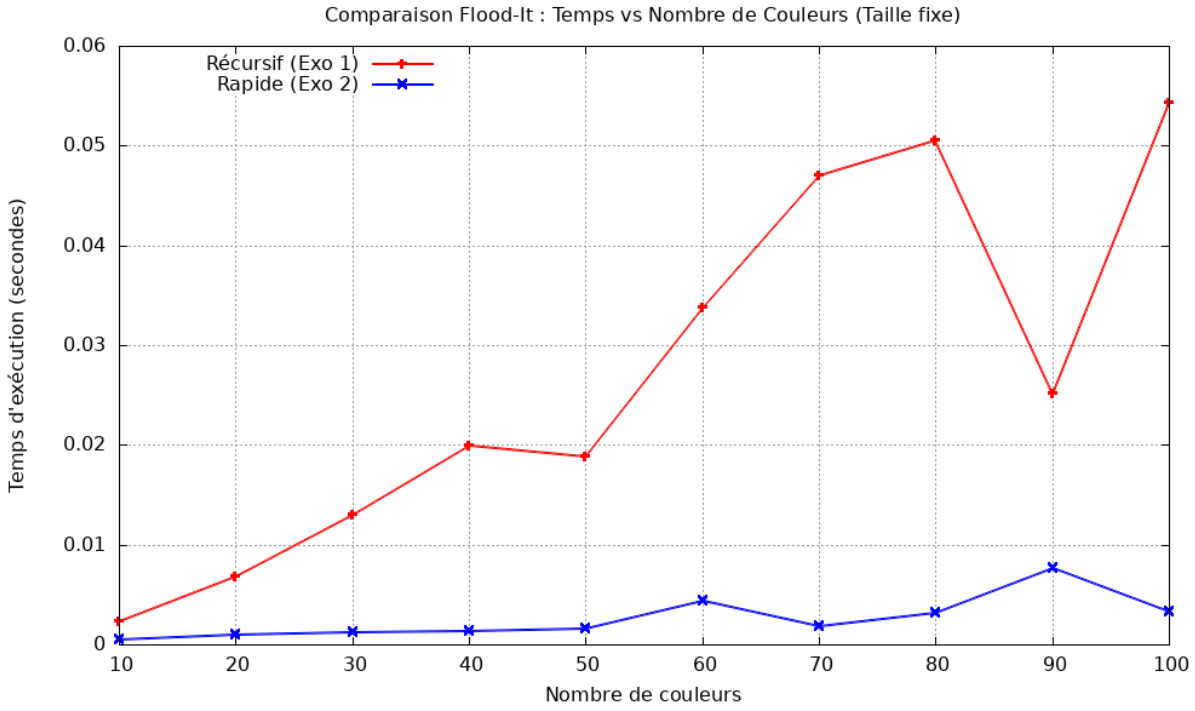
Lors des tests de montée en charge, j'ai constaté que l'algorithme récursif échouait systématiquement pour des grilles dépassant une taille d'environ 300x300.

- **Cause :** Chaque case visitée récursivement ajoute un cadre dans la pile d'exécution. Pour de grandes grilles, la profondeur de récursion sature la mémoire allouée à la pile, provoquant un *Stack Overflow* (dépassement de pile).
- **Solution :** L'approche itérative (Exo 2) utilise le tas via des listes chaînées (`ListeCase`), permettant de traiter des grilles 1000x1000 sans plantage.

Analyse des courbes : (comparaison_algos.png / comparaison_algos_couleurs.png)



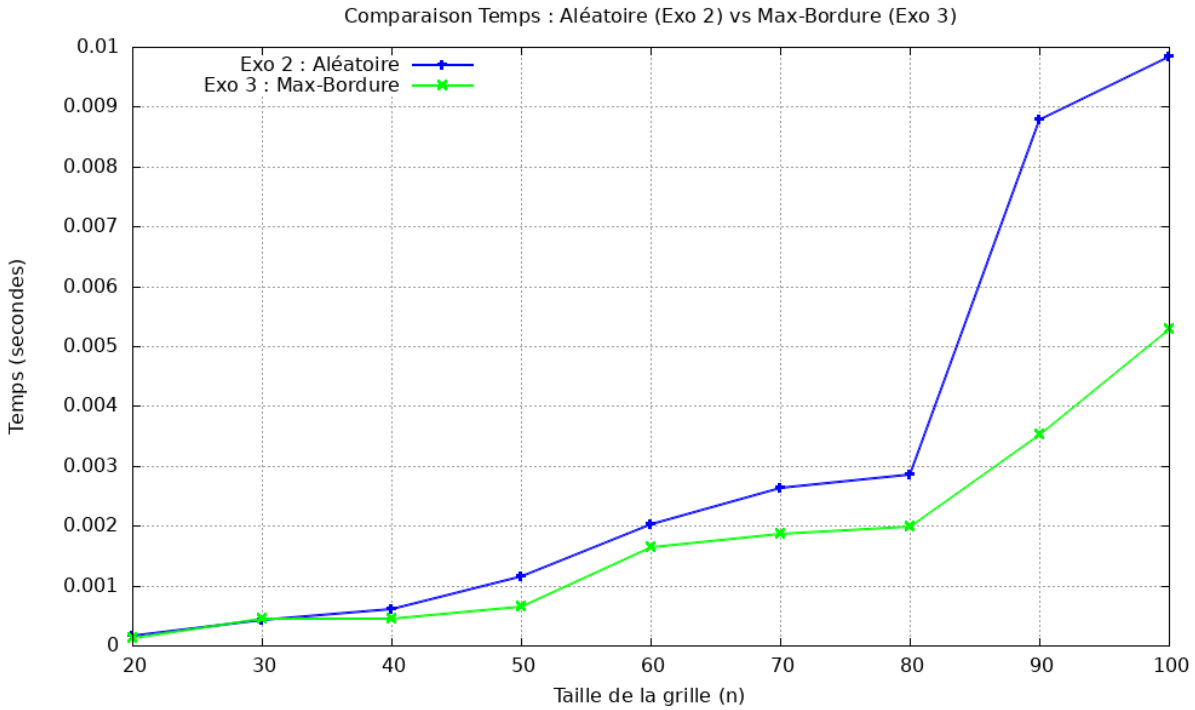
- **Temps vs Dimension :** La courbe rouge (Exo 1) s'arrête brusquement vers n = 300 illustrant le crash. La courbe bleue (Exo 2) montre une complexité polynomiale stable. Bien que l'algo rapide ait des pics, il reste fonctionnel sur de grandes instances.



Temps vs Couleurs : Sur une taille fixe, l'algo récursif (rouge) est nettement plus lent que l'itératif (bleu). L'algorithme rapide optimise la recherche des voisins en ne parcourant que la bordure, tandis que le récursif reparcourt souvent toute la zone Zsg.

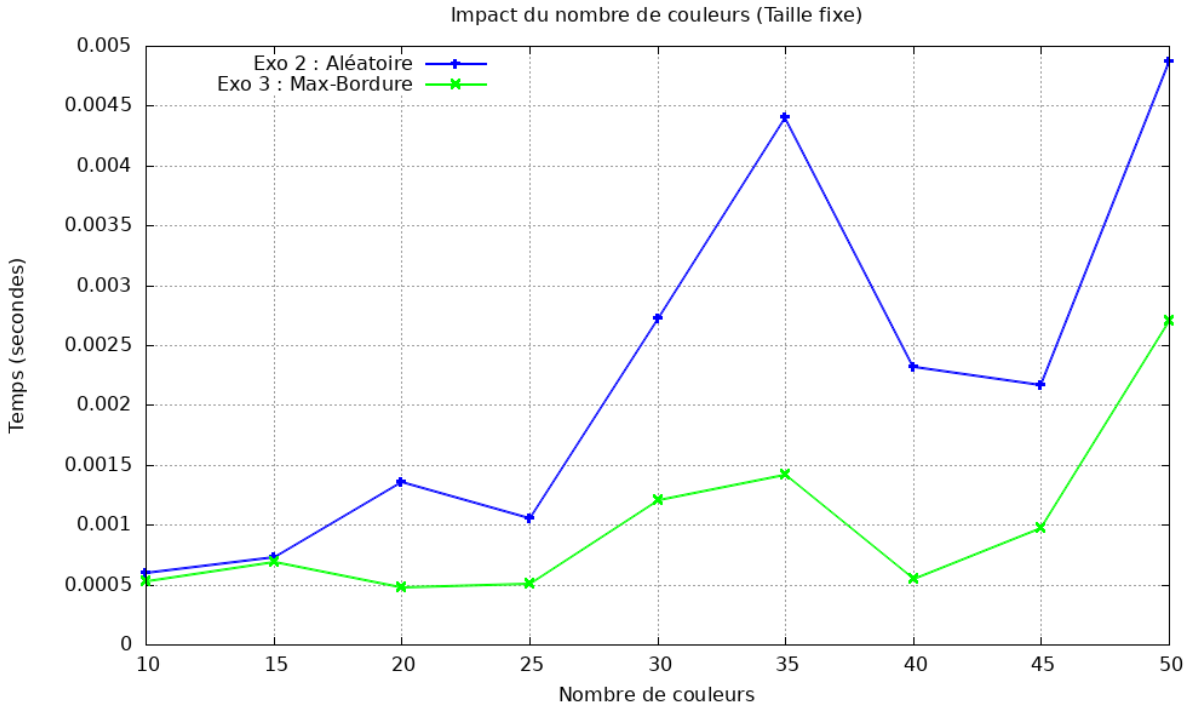
2.2. Exercice 2 (Aléatoire) vs Exercice 3 (Max-Bordure)

Analyse des Courbes : (comparaison_2_3_taille.png / comparaison_2_3_couleur.png)



Impact de la Taille : La courbe verte (Exo 3) reste en dessous de la bleue (Exo 2) pour les grandes tailles.

Interprétation : La stratégie Max-Bordure prend du temps à "réfléchir" (scanner la bordure à chaque tour), ce qui est coûteux. Cependant, elle réduit drastiquement le nombre total de coups pour finir le jeu.



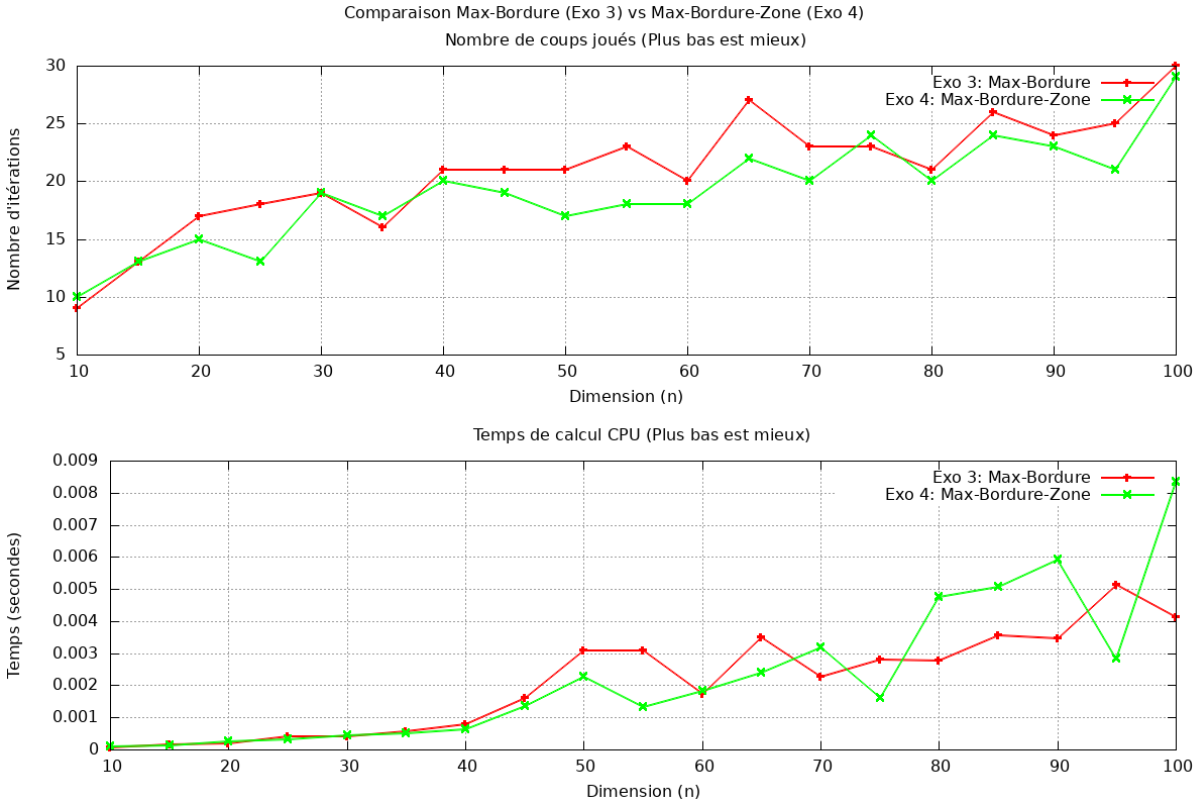
Impact des Couleurs :

L'écart est flagrant. Plus il y a de couleurs, plus la probabilité que l'aléatoire choisisse la "bonne" couleur chute. La courbe bleue explose, tandis que Max-Bordure reste stable car il élimine le hasard.

2.3. Exercice 3 (Max-Bordure) vs Exercice 4 (Max-Bordure-Zone)

L'exercice 4 est encore plus intelligent : au lieu de compter juste les pixels en contact (bordure), on compte **toute la tache de couleur** connectée à cette bordure.

Analyse des Courbes : (comparaison_strategies.png)



- **Nombre de coups (Graphe du haut) :** La courbe verte (Exo 4) est systématiquement inférieure à la rouge (Exo 3). L'algorithme "Zone" est plus intelligent : il "voit" plus loin et privilégie les couleurs qui ouvrent de grandes zones, même si leur bordure immédiate est petite.
- **Temps CPU (Graphe du bas) :** Les temps sont très proches, parfois légèrement supérieurs pour l'Exo 4. C'est le compromis classique : une meilleure décision coûte plus cher à calculer (parcours en profondeur des zones adjacentes), mais on gagne sur la longueur totale de la partie.

3. Proposition pour l'Exercice 5

Pour l'exercice 5, j'ai implémenté une stratégie qui ne se contente pas de l'état actuel, mais qui **simule** le coup pour estimer le gain réel.

Principe de la fonction `estime_taille_zone` :

1. Pour chaque couleur candidate, je simule une inondation virtuelle.
2. J'utilise un marqueur spécial (`VISITE_SIMU = -9`) dans la matrice `App` pour compter les gains sans modifier définitivement la grille de jeu.
3. Après l'estimation, une étape de nettoyage remet les marqueurs à leur état d'origine (`NON_RENCONTRE` ou couleur d'origine).

Résultat : Cette stratégie, bien que plus coûteuse en temps de calcul (car elle simule tous les coups possibles à chaque étape), offre le nombre de coups le plus optimal parmi toutes les stratégies testées.

4. Difficultés Rencontrées

Gestion de la Mémoire : La manipulation des listes chaînées (`ListeCase`) et des structures complexes (`S_Zsg`) a nécessité une rigueur particulière pour éviter les fuites de mémoire (`free`), notamment lors de la destruction des listes de bordures à chaque tour.

Compilation Multiple : Gérer plusieurs exécutables (le jeu principal et les différents tests de performance) a nécessité la création d'un Makefile modulaire.

5. Guide d'Utilisation

Voici la procédure pour compiler et exécuter le projet.

5.1. Installation

1. Entrez dans le dossier : `cd Flood-It-C`

5.2. Compilation

Utilisez le `Makefile` fourni pour compiler le jeu et les tests :

- Pour tout compiler : `make (voici le résultat avec make)`

```
leo@DESKTOP-FSDADRC:~/Code_enonce/Code_enonce$ ls
API_Gene_instance.c  API_Grille.o        Flood-It.o        Fonctions_exo4.c  Liste_case.h  S_Zsg.o           Test_Temps.c      comparaison2.gp
API_Gene_instance.h  Entete_Fonctions.h  Fonctions_exo1.c  Fonctions_exo4.o  Liste_case.o  Test_Strategies   Test_Temps.o      courbes.gp
API_Gene_instance.o  Flood-It            Fonctions_exo1.o  Fonctions_exo5.c  Makefile      Test_Strategies.c comp_exo2_exo3    courbes_couleurs.gp
API_Grille.c         Flood-It.c          Fonctions_exo3.c  Fonctions_exo5.o  S_Zsg.c       Test_Strategies.o comp_exo2_exo3.c  plot_strategies.gp
API_Grille.h         Flood-It.c~         Fonctions_exo3.o  Liste_case.c      S_Zsg.h       Test_Temps        comp_exo2_exo3.o
```

- Pour le jeu seul : `make FloodIt`
- Pour les tests de performance : `make Test_Temps Test_Strategies comp_exo2_exo3`

5.3. Exécution du Jeu

Pour lancer une partie, utilisez la commande suivante : `./FloodIt <dim> <nbcl> <difficulte> <graine> <algo> <aff>`

```
leo@DESKTOP-FSDADRC:~/Code_enonce/Code_enonce$ ./Flood-It 500 10 2 20 3 1
224 essais
1.696997
```

- **Exemple :** `./FloodIt 500 10 2 20 3 1` (Taille 500, 10 couleurs, difficulté 2 , graine 20, Algo Max-Bordure (exo 3), Affichage activé).
- **Codes Algo :** 1=Récursif, 2=Rapide, 3=Max-Bordure, 4=Max-Bordure-Zone, 5=Estimation.

5.4. Génération des Comparaisons

Pour reproduire les courbes présentes dans ce rapport :

1. Exécutez les scripts de test : `./Test_Temps` (Génère la comparaison entre l'exo 1 et l'exo 2 en générant 4 fichiers textes : donnees_rapide.txt et donnees_rapide_couleur.txt , donnees_rec.txt , donnees_rec_couleur.txt)

   `./comp_exo2_exo3` (Génère les données comparatives entre l'Exo 2 et l'Exo 3 sous forme de 4 fichiers texte : data_exo2_taille.txt, data_exo2_couleurs.txt , data_exo3_taille.txt , data_exo3_couleurs.txt)

   `./Test_Strategies` (Génère les données comparatives entre l'Exo 3 vs 4 en générant un unique fichiers texte : donnees_strategies.txt)

2. Lancez les scripts Gnuplot :

   `gnuplot courbes.gp courbes_couleurs`.gp (pour les graphes de la comparaison entre l'exo 1 et l'exo 2 , comparaison_algos.png et comparaison_algos_couleurs.png).

   `gnuplot comparaison2.gp (pour avoir les graphes de comparaison entre l'exo 2 et l'exo 3 , comparaison_2_3_couleurs.png et comparaison_2_3_taille.png).`

   `gnuplot plot_strategies.gp (pour avoir le graphe de comparaison entre l'exo 3 et l'exo 4 , comparaison_strategies.png).`

Les images `.png` seront générées dans le dossier du projet.